

RepairAgent: An Autonomous, LLM-Based Agent for Program Repair (ICSE 2025)

Raphael Torres Cavalcanti Ferreira

Rubens Nascimento de Lima

Thaís Neves de Souza

Vinicius Resende Barbosa

Tópicos

1. Exploração do artigo base
2. Metodologia da Replicação
3. Comparação de Resultados
4. Conclusão

Exploração do Artigo Base

Visão Geral da Pesquisa

- O primeiro agente autônomo baseado em Large Language Models (LLMs) **totalmente focado em Automated Program Repair (APR)**.
- Diferente de abordagens tradicionais que usam prompts estáticos (one-shot), o RepairAgent **simula o comportamento de um desenvolvedor humano**.
- O modelo intercala as ações de reunir informações, buscar ingredientes para o patch e validar correções com base no feedback contínuo.

A Máquina de Estados Finitos

- O agente analisa os relatórios de falha da suíte de testes e o log de erros para formular uma hipótese inicial sobre a causa raiz.
- O modelo navega pelo repositório, buscando arquivos, extraíndo assinaturas de métodos e compreendendo a estrutura do código afetado.
- O agente propõe alterações, aplica os patches no código e executa os testes. Se falhar, ele usa o log de erro para voltar aos estados anteriores e refinar a solução.

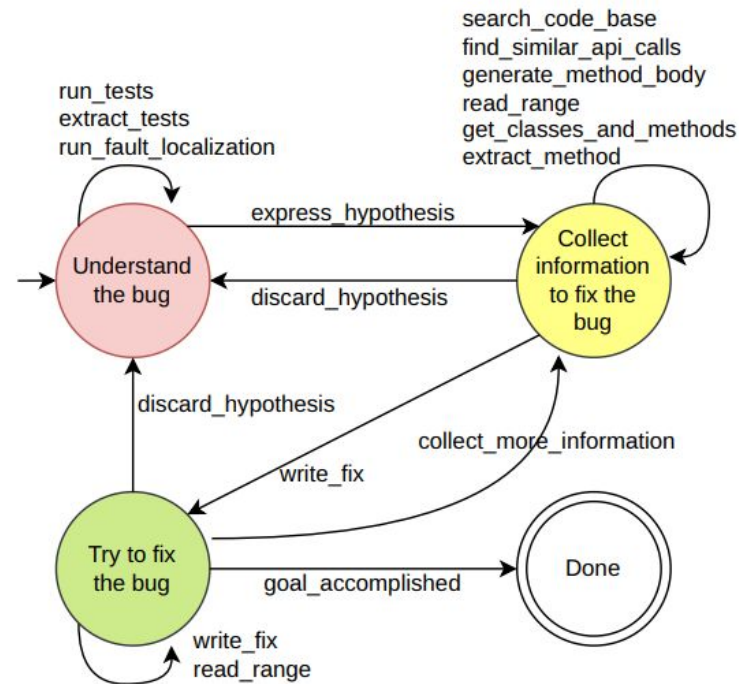


Fig. 2: State machine to guide selection of tools.

O Kit de Ferramentas (Tool-Use)

- **Navegação:** Comandos para listar diretórios e buscar palavras-chave específicas em todo o repositório.
- **Extração:** Capacidade de ler linhas específicas, inspecionar classes e métodos para entender o escopo léxico.
- **Interação:** Aplicação de patches cirúrgicos e execução autônoma do comando de teste (recebendo o stack trace exato em caso de falha).

Prompt Dinâmico

- **Histórico de Contexto:** Para evitar que o modelo fique preso em loops e repita os mesmos erros, o sistema atualiza o prompt dinamicamente a cada ciclo.
- **Memória Agêntica:** O LLM recebe constantemente um resumo das ferramentas que já invocou, os retornos que obteve e os patches que já falharam, forçando a evolução das hipóteses.

Resultados e Impacto

- Validado no **Defects4J**, um benchmark de bugs reais em projetos Java.
- O RepairAgent conseguiu corrigir com sucesso **164 bugs** de forma autônoma.
- Dentre os acertos, **39 bugs** consertados pelo modelo **não haviam sido resolvidos** por nenhuma das principais técnicas anteriores de reparo automático.
- Além de resultados superiores, apresentou melhor desempenho em bugs mais complexos, incluindo correções envolvendo múltiplas linhas e arquivos.

Bugs Resolvidos no Artigo por Projeto

TABLE III: Results on Defects4J.

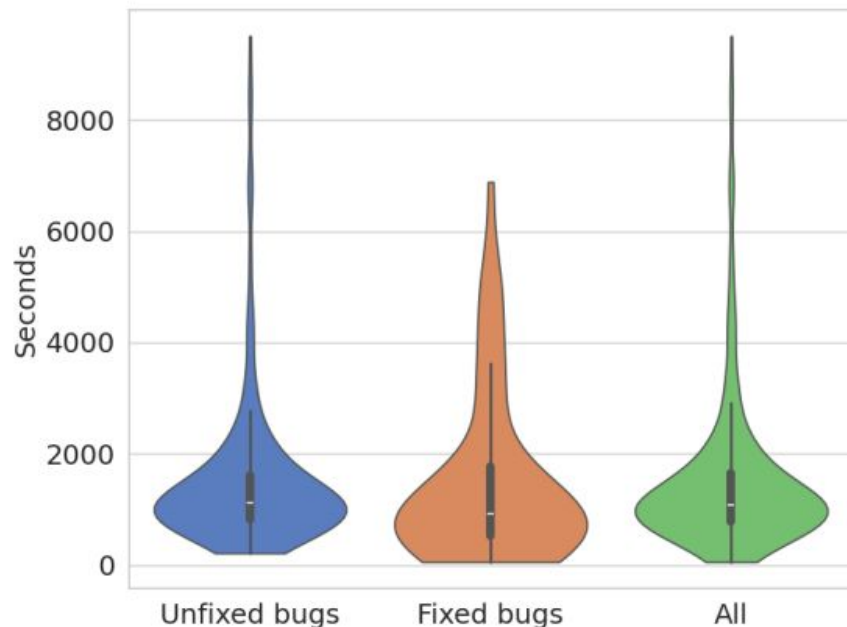
Project	Bugs	Plausible	Correct	ChatRepair	ITER	SelfAPR
Chart	26	14	11	15	10	7
Cli	39	9	8	5	6	8
Closure	174	27	27	37	18	20
Codec	18	10	9	8	3	8
Collections	4	1	1	0	0	1
Compress	47	10	10	2	4	7
Csv	16	6	6	3	2	1
Gson	18	3	3	3	0	1
JacksonCore	26	5	5	3	3	3
Jackson databind	112	18	11	9	0	8
JacksonXml	6	1	1	1	0	1
Jsoup	93	18	18	14	0	6
JXPath	22	0	0	0	0	1
Lang	63	17	17	21	0	10
Math	106	29	29	32	0	22
Mockito	38	6	6	6	0	3
Time	26	3	2	3	2	3
Defects4Jv1.2	395	96	74	114	57	64
Defects4Jv2	440	90	90	48	—	46
Total	835	186	164	162	57	110

Tipos de Bugs Resolvidos no Artigo

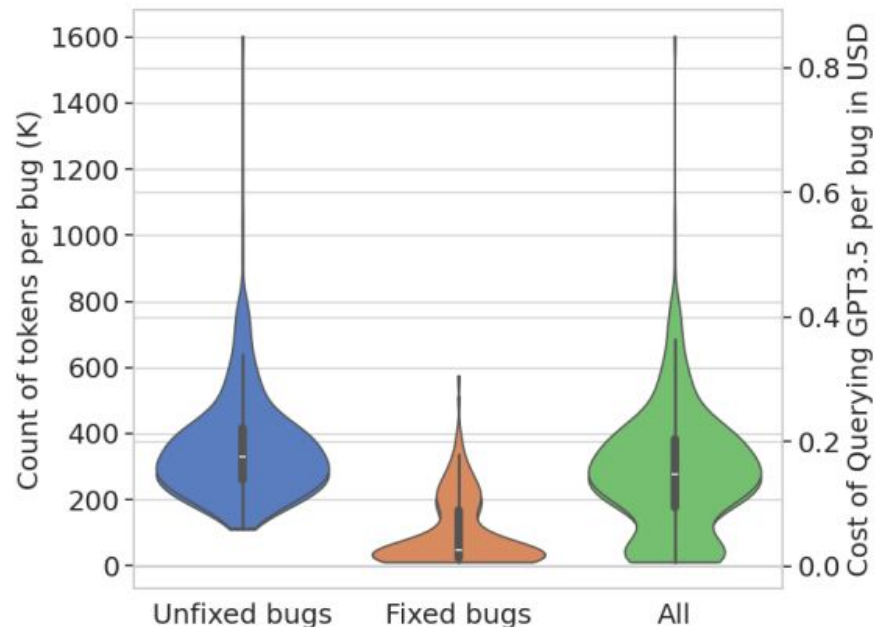
TABLE IV: Distribution of fixes by location type

Bug type	RepairAgent	ChatRepair	ITER	SelfAPR
Single-line	115	133	36	83
Multi-line*	46	29	14	24
Multi-file	3	0	4	3

Distribuição de Tempo e Tokens gastos no Artigo



(a) Time.



(b) Tokens/money consumption.

Fig. 9: Distribution of cost metrics per bug (time, number of token, and monetary costs).

Metodologia de Replicação

Metodologia de Replicação e Adaptação

- Fork do Repair Agent
- Adaptações para a comunicação com os Modelos
- Suporte a modelos locais/API via Ollama
- Debug interativo

Seleção dos Modelos Locais

- Testes exploratórios com diferentes modelos
- Seleção do GPT-OSS (20b) como modelo para os testes.
- Uso de modelos quantizados.

Modelo	Observação
GPT-OSS 20B	Melhor equilíbrio entre execução e comportamento como agente.
Gemma 3 4B	Dificuldade em seguir comandos e formato esperado
Gemma 4 E4B (8B)	Tempo elevado de thinking e recorrência de timeouts

Controle de Execução do Agente

- Redução no limite de ciclos
- Tratamento de timeouts
 - Até 3 tentativas
 - Incremento da Temperatura
 - Desabilitar o “*thinking*” do modelos
- Execução sequencial

Configuração	Artigo Base	Replicação
Limite de Ciclos	40	15
Timeout por resposta	Não informado	3 minutos
Execução	API (paralela)	Local (sequencial)
Modelo	GPT-3.5	GPT-OSS 20B

Controle de Execução do Agente

Métrica	Significado
Patch Plausível	Se possuiu um patch de alteração que passou em todos os testes
Ciclos	Quantidade de interações do agente.
Tokens	Quantidade de tokens gerados e lidos durante todo o processo
Tempo	Duração da Execução (modelo + ferramentas)
Complexidade	Linhas e arquivos alterados

Item	Definição
Benchmark	Defects4J
Amostragem	Subset estratificado
Limite	Até 10 bugs por projeto
Comparação	Percentual/normalizada

Resultados

Porcentagem de Bugs Resolvidos (Tabela Geral)

OBS:

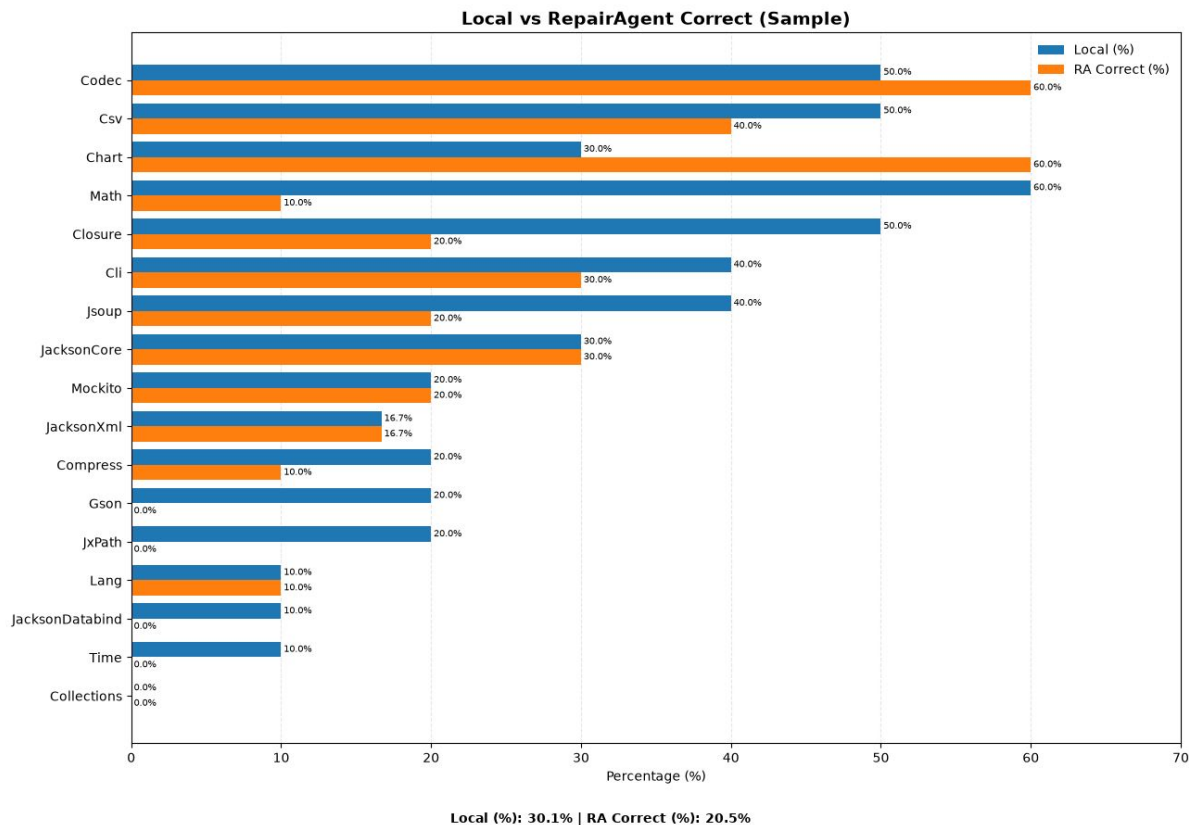
% LocalAgent= 5.6%
(Não foi executado em todos os bugs; Amostra = 18.68% do total)

~% RepairAgent= 20.5%
(Enviesada pela amostra, mas próxima da total; Amostra estratificada por projetos)

~% RA Plausible= ?
(Seria 28.8% no caso extremo)

Project	Local (~%)	Local (%)	RA Plausible* (%)	RA Correct (~%)	RA Correct (%)
Chart	30.0% (3/10)	11.5% (3/26)	53.8% (14/26)	60.0% (6/10)	42.3% (11/26)
Cli	40.0% (4/10)	10.3% (4/39)	23.1% (9/39)	30.0% (3/10)	20.5% (8/39)
Closure	50.0% (5/10)	2.9% (5/174)	15.5% (27/174)	20.0% (2/10)	15.5% (27/174)
Codec	50.0% (5/10)	27.8% (5/18)	55.6% (10/18)	60.0% (6/10)	50.0% (9/18)
Collections	0.0% (0/0)	0.0% (0/4)	25.0% (1/4)	0.0% (0/0)	25.0% (1/4)
Compress	20.0% (2/10)	4.3% (2/47)	21.3% (10/47)	10.0% (1/10)	21.3% (10/47)
Csv	50.0% (5/10)	31.2% (5/16)	37.5% (6/16)	40.0% (4/10)	37.5% (6/16)
Gson	20.0% (2/10)	11.1% (2/18)	16.7% (3/18)	0.0% (0/10)	16.7% (3/18)
JacksonCore	30.0% (3/10)	11.5% (3/26)	19.2% (5/26)	30.0% (3/10)	19.2% (5/26)
JacksonDatabind	10.0% (1/10)	0.9% (1/112)	16.1% (18/112)	0.0% (0/10)	9.8% (11/112)
JacksonXml	16.7% (1/6)	14.3% (1/7)	14.3% (1/7)	16.7% (1/6)	14.3% (1/7)
Jsoup	40.0% (4/10)	4.3% (4/93)	19.4% (18/93)	20.0% (2/10)	19.4% (18/93)
JXPath	20.0% (2/10)	9.1% (2/22)	0.0% (0/22)	0.0% (0/10)	0.0% (0/22)
Lang	10.0% (1/10)	1.6% (1/63)	27.0% (17/63)	10.0% (1/10)	27.0% (17/63)
Math	60.0% (6/10)	5.7% (6/106)	27.4% (29/106)	10.0% (1/10)	27.4% (29/106)
Mockito	20.0% (2/10)	5.3% (2/38)	15.8% (6/38)	20.0% (2/10)	15.8% (6/38)
Time	10.0% (1/10)	3.8% (1/26)	11.5% (3/26)	0.0% (0/10)	7.7% (2/26)
Total	30.1% (47/156)	5.6% (47/835)	21.2% (177/835)	20.5% (32/156)	19.6% (164/835)

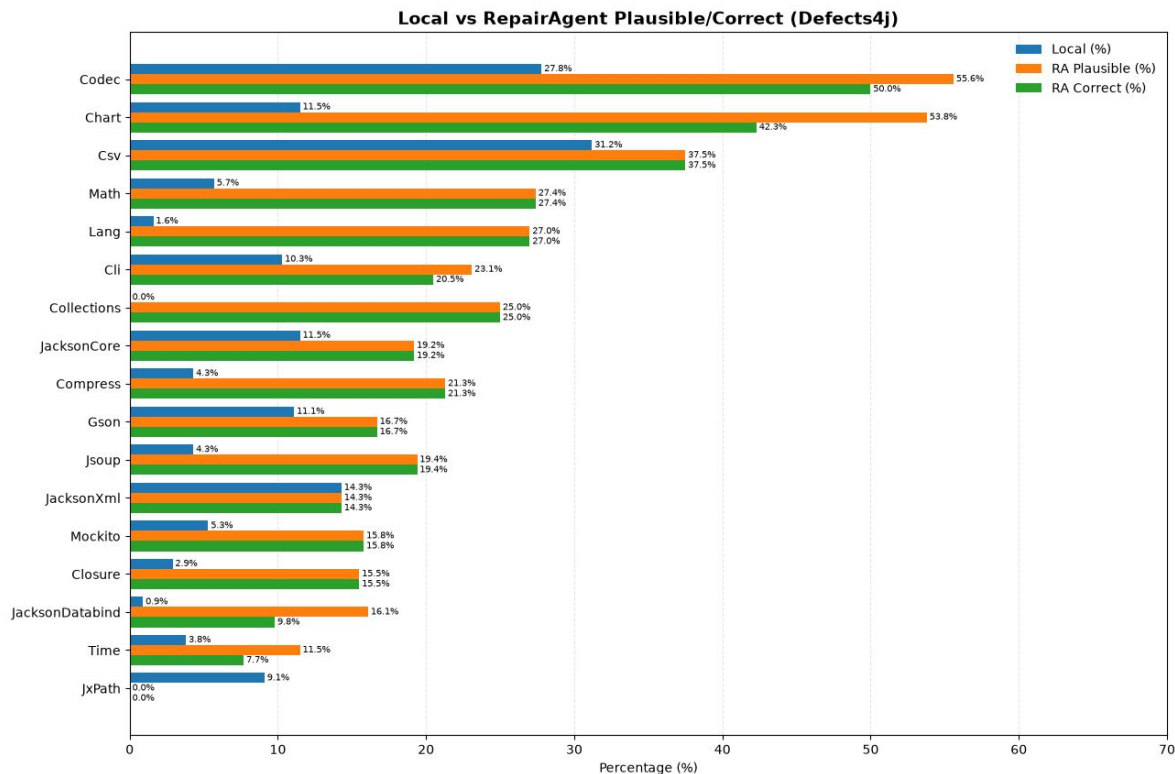
Porcentagem de Bugs Resolvidos por Projeto na Amostra



OBS:

RA Plausible (caso extremo) = 28,84%

Porcentagem de Bugs Resolvidos por Projeto no dataset*



Local (%): 5.6% | RA Plausible (%): 21.2% | RA Correct (%): 19.6%

Fixes Adicionados (Local vs RepairAgent Correct)

Project	Local Added Count	Local Added Fixes	Repair Added Count	Repair Added Fixes
Chart	1	10	4	4, 5, 8, 9
Cli	2	4, 7	1	8
Closure	3	2, 5, 2007	0	-
Codec	2	5, 10	3	2, 4, 2008
Compress	2	6, 7	1	1
Csv	2	5, 6	1	2
Gson	2	5, 6	0	-
JacksonCore	1	9	1	8
JacksonDatabind	1	1	0	-
Jsoup	2	4, 10	0	-
JXPath	2	5, 8	0	-
Lang	1	4	1	10
Math	5	3, 4, 5, 8, 10	0	-
Mockito	1	9	1	5
Time	1	8	0	-
Total	28	-	13	-

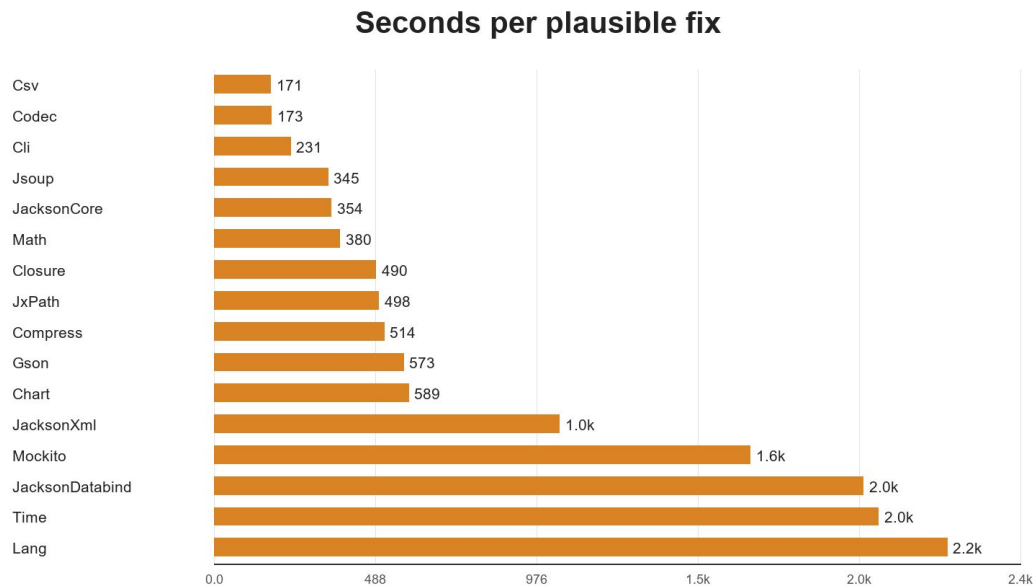
OBS: RA Plausible (caso extremo) = 26 fixes adicionados

Fixes por Tipo de Localização

Bug-Type	Local Agent	Repair Agent
Single-line	13 (27.7%)	115 (70.1%)
Multi-Line	25 (53.2%)	46 (28%)
Multi-file	2 (4.3)	3 (1.8%)

OBS: 7 bugs não identificados na análise local

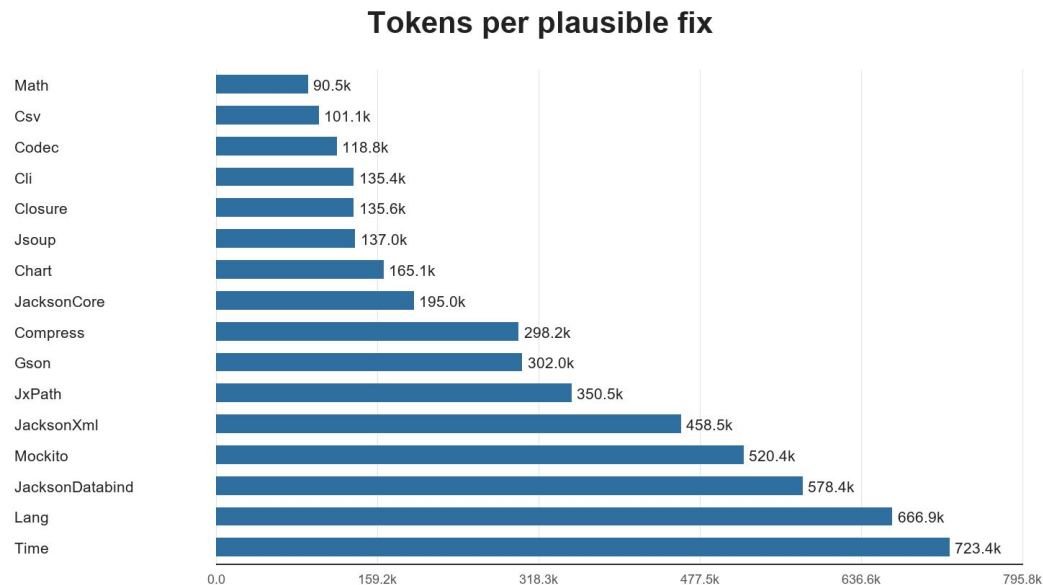
Tempo médio gasto por bug fix por projeto



Projects with zero plausible successes are omitted.

OBS: Mediana do Repair Agent = 920s

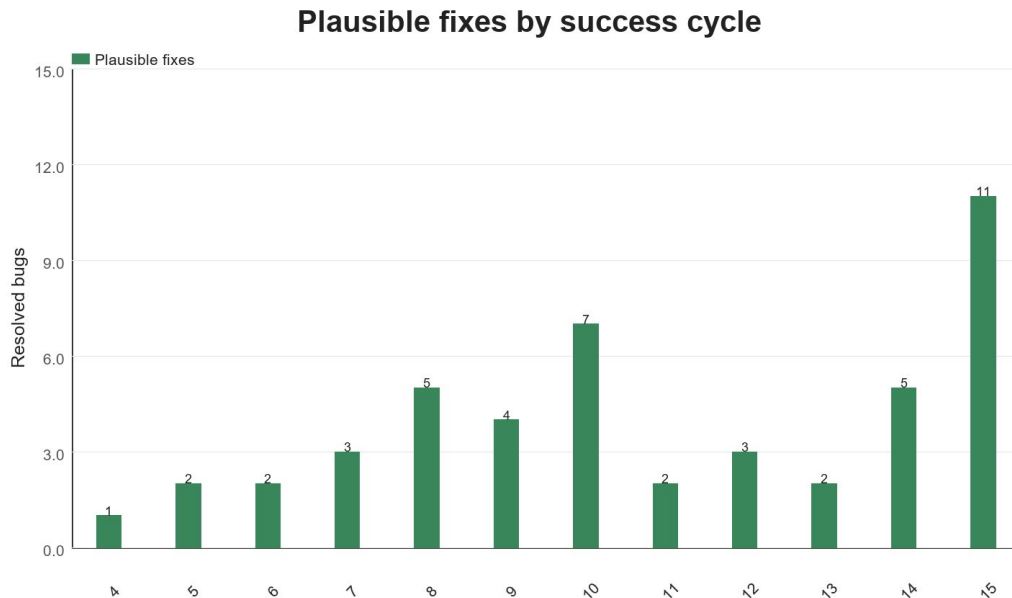
Tokens gastos em média por bug fix por projeto



Projects with zero plausible successes are omitted.

OBS: Mediana do Repair Agent $\approx$ 270k tokens

Ciclos usados até a geração de uma resolução plausível



X-axis starts at the earliest cycle with a plausible fix and ends at the latest successful cycle.

Análise Qualitativa

- Para avaliar a eficácia e o comportamento do RepairAgent com um modelo local, adotou-se uma abordagem de análise qualitativa.
- Cada integrante ficou responsável por analisar um conjunto de projetos do benchmark.
- Em cada projeto, isolamos e confrontamos diretamente dois cenários distintos, um case de sucesso (onde o agente gerou um patch plausível que eliminou as falhas) e um case de falha (onde a execução foi interrompida sem a resolução do bug).

Análise Qualitativa

- Durante a leitura dos arquivos de log, o foco concentrou-se em mapear as dores do agente, identificando motivos de travamento, loops de repetição, erros de compilação, problemas com o parser de JSON e falhas de comunicação com o middleware.
- Investigou-se como o modelo se comportava frente às categorias dos bugs, comparando a eficiência na correção de linhas contra a capacidade de criar novos blocos de código.

Conclusão

Principais Conclusões

- Os resultados mostram que o agente possui capacidade para **compreensão** e **localização** da causa raiz dos defeitos de software, mesmo com um modelo local
- O maior índice de falhas (bugs não resolvidos) não decorreu da incapacidade de compreender a lógica do código, mas sim de **dificuldade de interação com o ecossistema agêntico**
- O agente demonstra sucesso em **modificações mais locais**, mas encontra dificuldades quando o bug exige **alterações complexas em diferentes arquivos**